3. Any partial response that has been defined is cleared.
4. The `ExceptionHandlerMiddleware` overwrites the request path with the provided error-handling path.
5. The middleware sends the request back down the pipeline, as though the original request had been for the error-handling path.
6. The middleware pipeline generates a new response as normal.
7. When the response gets back to `ExceptionHandlerMiddleware`, it modifies the status code to a 500 error and continues to pass the response up the pipeline to the web server.

One of the main advantages of reexecuting the pipeline for Razor Page apps is the ability to have your error messages integrated into your normal site layout, as shown in figure 4.17. It's certainly possible to return a fixed response when an error occurs without reexecuting the pipeline, but you wouldn't be able to have a menu bar with dynamically generated links or display the current user's name in the menu, for example. By reexecuting the pipeline, you ensure that all the dynamic areas of your application are integrated correctly, as though the page were a standard page of your site.

NOTE You don't need to do anything other than add `ExceptionHandlerMiddleware` to your application and configure a valid error-handling path to enable reexecuting the pipeline, as shown in figure 4.18. The middleware will catch the exception and reexecute the pipeline for you. Subsequent middleware will treat the reexecution as a new request, but previous middleware in the pipeline won't be aware that anything unusual happened.

Reexecuting the middleware pipeline is a great way to keep consistency in your web application for error pages, but you should be aware of some